Theoretical and Architectural Foundations of Agentic AI: A System-Theoretic Review of Design Patterns and Frameworks

The transition within artificial intelligence from static generative models to autonomous agentic systems represents a fundamental shift in the design of intelligent software. While generative AI focuses on mapping fixed inputs to singular outputs, agentic AI introduces a dynamic paradigm where systems are designed to actively modify the state of their environment through continuous cycles of perception, reasoning, and action.[1] In the 2024–2025 landscape, agentic systems have moved beyond mere supporting tools to become active participants in complex decision-making processes across the digital economy, scientific discovery, and robotics.[1] This evolution is underpinned by the integration of advanced cognitive architectures that incorporate long-term memory, multi-stage planning, and persistent identity through profiling.[1] As these systems gain autonomy, the architectural requirements for reliability, safety, and efficiency become increasingly stringent, necessitating a formal taxonomy of design patterns that can be systematically applied to achieve robust, goal-directed behavior.[2]

A System-Theoretic Decomposition of Agentic Architectures

The architectural complexity of modern agentic systems requires a principled methodology that goes beyond ad-hoc prompting strategies. A novel system-theoretic framework deconstructs an agentic AI system into five core functional subsystems that interact to produce autonomous behavior: Reasoning & World Model, Perception & Grounding, Action Execution, Learning & Adaptation, and Inter-Agent Communication.[3] This layered organization provides a theoretical foundation for the construction and analysis of agents as proactive, contextual entities.[3]

Perception and Grounding Subsystems

Perception has evolved from the simple processing of raw text to sophisticated multimodal integration, encompassing visual, audio, and environmental signals.[1] This enables agents to interpret complex graphical user interfaces (GUIs) and navigate cyber-physical environments.[1] Grounding is the process of mapping linguistic constructs to specific environmental states, a critical requirement for ensuring that an agent's reasoning remains anchored in reality.[3] Hallucination and factual errors often arise when this grounding is weak, highlighting the need

for robust world modeling that maintains consistency across long horizons.[3]

Cognitive Dimensions and the Modular Taxonomy

A holistic taxonomy of the agentic ecosystem decomposes large language model (LLM) based agents into six modular dimensions: Core Components, Cognitive Architecture, Learning, Multi-Agent Systems, Environments, and Evaluation.[1] Within the core components, profiling defines the agent's identity through system prompts and dynamic roles, ensuring behavioral consistency.[1] Cognitive architecture focuses on the higher-level logic of planning and reflection, while the learning dimension addresses how agents adapt through experience.[1]

Dimension	Primary Components	Evolutionary Shift
Core Components	Perception, Memory, Action, Profiling	From text-only to multimodal; from transient to persistent memory [1]
Cognitive Architecture	Planning, Reflection	From linear execution to iterative search and self-critique [1]
Learning	Feedback loops, Parameter-efficient fine-tuning	From static weights to dynamic context-based adaptation [1]
Multi-Agent Systems	Collaboration, Negotiation, Governance	From isolated actors to organized, communicative communities [1]
Environments	Digital (Browsers), Cyber-physical (Robotics)	From closed-loop simulations to real-world deployment [1]

Formal Foundations: The BDI Model and Markovian Agency

The conceptual underpinnings of agentic AI draw heavily from classical AI theories, most notably the Belief-Desire-Intention (BDI) model. Formulated by Michael Bratman, the BDI theory posits that an agent's mental state consists of its informational state (Beliefs), motivational state (Desires), and deliberative commitments (Intentions).[10] In modern LLM-based agents, these states are often implicitly managed through complex prompts and context windows. A formal

definition of an LLM agent is characterized by the tuple $A = (M, E, S, A, \pi)$, where M is the underlying language model, E is the environment, S is the state space, A is the action space, and $\pi : S \to A$ is the policy.[7]

The agentic control loop—an iterative cycle of sensing, deliberating, and acting—mirrors the BDI deliberation cycle.[11] When an agent "drifts" off-task, it signifies a failure in its intention mechanism, a problem often addressed in prompt engineering by explicitly reinforcing original goals.[12] Historically, symbolic systems like SOAR and ACT-R addressed these issues through activation-based retrieval and automatic sub-goal creation, concepts that have been "reinvented" in modern neural patterns such as RAG and Reflexion.[12] The emergence of "synthetic purposiveness" in LLM agents suggests that agency is being reconstituted within computational substrates rather than being a mere extension of classical models.[13]

Foundational Reasoning Patterns

Reasoning-enhanced agents focus on improving decision-making quality through structured prompting and search algorithms.[7] These patterns transition from linear sequences to complex graph-based deliberations, allowing for nuanced problem decomposition.[7]

Chain-of-Thought (CoT) and Self-Consistency

Chain-of-Thought (CoT) prompting involves eliciting step-by-step reasoning from the model, enabling it to break down complicated tasks into manageable logical units.[7] This paradigm has demonstrated significant accuracy gains in arithmetic and symbolic reasoning tasks, such as improving PaLM-540B's success rate on the GSM8K benchmark from 17.9% to 56.9%.[7] Zero-shot CoT, which utilizes the trigger phrase "Let's think step by step," further simplifies this by eliciting reasoning behavior without task-specific examples.[7] To enhance robustness, the Self-Consistency pattern samples multiple diverse reasoning paths and selects the most frequent answer through majority voting, leveraging the principle that consistency across different strategies provides stronger evidence for correctness.[7]

The ReAct Pattern: Synergizing Reasoning and Acting

The ReAct (Reason + Act) pattern is a foundational architecture that interleaves internal reasoning with external interactions.[1] Unlike purely latent CoT, ReAct generates explicit rationales (thoughts) that condition actions, which in turn produce observations from the environment.[1] A ReAct trajectory is formally defined as $\tau = \{o_1, z_1, a_1, o_2, z_2, a_2, \dots\}$, where rationales z_t guide tool calls or API executions a_t, and environmental observations o_t update the agent's state.[1] This feedback loop improves groundedness by allowing the agent to adapt its plan based on real-world results.[1] However, ReAct remains vulnerable to myopia and

error propagation, where an early mistake leads to unproductive loops.[1]

Iterative Refinement and Reflective Design Patterns

Reflective patterns incorporate a self-correction layer, allowing agents to evaluate and improve their own work autonomously.[17] These patterns are particularly valuable in domains where the cost of error is high and quality takes precedence over speed.[20]

The Reflexion Framework: Verbal Reinforcement Learning

Reflexion, proposed by Shinn et al., is a paradigm for "verbal" reinforcement that parameterizes a policy as an agent's memory encoding paired with specific LLM parameters.[21] Instead of updating model weights, Reflexion uses linguistic feedback to optimize behavior iteratively.[19] The architecture consists of three core components:

1. **Actor:** The agent generating actions and thoughts (often utilizing ReAct or CoT).[21]
2. **Evaluator:** A module that scores the Actor's output, providing a reward signal based on heuristics or LLM-based assessment.[21]
3. **Self-Reflection:** An LLM that converts reward signals and trajectories into verbal critiques, which are stored in episodic memory to guide future attempts.[21]

Reflexion has demonstrated impressive results, such as reaching a 91% pass@1 accuracy on the HumanEval coding benchmark, surpassing the 80% baseline of GPT-4.[19] By storing reflections in long-term memory, the agent carries a log of lessons learned that bias its next actions toward better outcomes.[19]

Self-Refine and Multi-Pass Iteration

The Self-Refine pattern involves the agent critiquing its own response and re-executing the task to improve the output.[14] This pattern is effective across diverse tasks, from dialogue generation to mathematical reasoning, showing improvements of up to 20 percentage points.[24] In production environments, this is often implemented as a "reflect-and-critique" loop, where the agent generates an initial draft, switches to a reviewer persona to identify flaws, and then generates a final revised version.[17]

Pattern	Mechanism of Improvement	Target Use Case	Key Constraint
Reflexion	Cross-episodic verbal reinforcement;	Long-horizon tasks; trial-and-error learning [21]	High latency due to multiple turns [18]

	linguistic memory buffer [22]		
Self-Refine	In-session iterative critique and generation [14]	Factual content; security audits; report writing [17]	Potential for infinite self-critique loops [20]
CRITIC	Verification against external tools and iterative refinement [14]	Knowledge-intensive tasks; factual verification [14]	Dependency on tool reliability [14]

Advanced Planning and Deliberative Search

To overcome the limitations of linear reasoning, advanced patterns incorporate explicit search over reasoning and action trajectories.[15] These patterns treat problem-solving as a tree exploration process, enabling the agent to consider multiple paths and backtrack when necessary.[7]

Tree of Thoughts (ToT) and Graph of Thoughts (GoT)

Tree of Thoughts (ToT) treats intermediate thoughts as search nodes and explores alternatives via breadth-first or depth-first search strategies.[1] Graph of Thoughts (GoT) further extends this by allowing arbitrary connections between steps, enabling the aggregation of ideas from different reasoning branches and the refinement of complex logical networks.[7] These patterns are designed for tasks that require "deliberate and thoughtful decision-making" beyond reflexive prompting.[15]

Language Agent Tree Search (LATS)

Language Agent Tree Search (LATS) is a unified framework that fuses LLM reasoning with Monte Carlo Tree Search (MCTS).[15] LATS repurposes LLMs as agents, value functions, and optimizers within a strategic search process.[29] The LATS operating cycle is defined by six operations: selection, expansion, evaluation, simulation, backpropagation, and reflection.[26]

The selection process is governed by the Upper Confidence Bound for Trees (UCT) formula:

$$UCT(s) = V(s) + w\sqrt{\frac{\ln N(p)}{N(s)}}$$

where $V(s)$ is the value function estimate at node s, $N(s)$ is the visit count for the node,

$N(p)$ is the visit count for the parent, and w is the exploration constant.[26]

LATS excels in environments providing external feedback, such as programming (HumanEval) and web navigation (WebShop), where it can test code or observe webpage content to inform its search.[15] By sampling multiple trajectories and constructing the best one through backpropagation of reflective scores, LATS significantly outperforms linear patterns like ReAct and Reflexion.[15]

Memory Architectures and Virtual Context Management

The finite context window of LLMs presents a major obstacle for agents performing long-horizon tasks.[30] Memory-augmented agents incorporate persistent context through sophisticated storage tiers and management logic.[7]

The MemGPT Paradigm: LLMs as Operating Systems

MemGPT (MemoryGPT) introduces virtual context management, a technique inspired by traditional operating systems that provide the illusion of extended memory via paging between RAM and disk.[30] In this architecture, memory is divided into three tiers:

1. **Main Context (Tier 1):** The LLM's fixed context window, containing system instructions, working context (scratchpad), and conversational history (FIFO queue).[33]
2. **External Context (Tier 2):** Secondary storage consisting of archival memory (a massive, searchable vector database) and recall storage (the complete conversation history).[30]
3. **Self-Directed Management:** The agent uses tool calls (memory editing) to actively decide what to store, summarize, or retrieve, essentially managing its own context window.[30]

MemGPT prioritizing precision over recall, treating strategic forgetting as an essential feature to avoid "context pollution"—the clogging of the context window with irrelevant information.[30] The transformation of episodic memories (specific events) into semantic memories (general knowledge) in MemGPT replicates human cognitive processes, enabling long-term personalization and contextual awareness.[30]

Execution Patterns: Tool-Augmented Autonomy

Tool-augmented agents extend LLM capabilities by interacting with external APIs, databases, and computational environments.[7] This allows agents to overcome inherent limitations such as parametric knowledge gaps and the inability to perform precise arithmetic.[7]

Toolformer and Self-Supervised Learning

The seminal work on Toolformer established that LLMs can teach themselves to use tools (calculators, search engines, calendars) via simple APIs in a self-supervised manner.[35] The model is trained to decide which API to call, when to call it, and how to incorporate the result into future token prediction.[37] A filtering criterion ensures that tool calls are only retained if their results reduce the model's perplexity on subsequent tokens.[35]

The CodeAct Paradigm: Code as Action Language

CodeAct proposes the use of executable Python code as a unified action language for agents.[39] Instead of task-specific JSON schemas, CodeAct allows agents to emit Python code that can:

- **Implement Complex Logic:** Use loops, conditionals, and assertions within a single action.[39]
- **Manage Data Flow:** Store intermediate results in variables for reuse across multiple steps.[39]
- **Leverage Ecosystems:** Access readily available Python packages for expanded tool capability.[39]

CodeAct's ability to interpret stack traces and self-debug significantly improves success rates on multi-tool benchmarks, outperforming JSON-based alternatives by up to 20% while requiring fewer interaction turns.[39]

Multi-Agent Systems and Collaborative Patterns

Multi-agent systems (MAS) involve multiple specialized agents coordinating to solve problems too complex for a single agent.[8] These systems transition from isolated actors to collaboration-centric approaches, utilizing diverse orchestration patterns.[43]

Orchestration Topologies: Hierarchical vs. Peer-to-Peer

Orchestration defines the structure of agent interaction and task delegation.[44]

1. **Centralized/Hierarchical Orchestration:** A central supervisor or manager agent distributes tasks to specialized workers and aggregates results.[16] This hub-and-spoke model is highly stable for enterprise applications where strict governance is required.[16] Frameworks like CrewAI model agents as "crew members" with backstories and roles (e.g., Coder, Researcher), where a manager agent can coordinate sequential or parallel execution.[46]
2. **Decentralized/Peer-to-Peer Orchestration:** Agents communicate directly through service discovery and negotiation protocols.[45] While more flexible, research indicates that independent agents working in parallel without coordination can amplify errors by up to

17.2x, compared to 4.4x in centralized systems.[48]

Communication Protocols and Language

Standardized protocols enable interoperability between heterogeneous agents.[49]

- **FIPA-ACL and KQML:** Landmark protocols that specify message structures and performative verbs like "request," "inform," and "propose".[49]
- **Model Context Protocol (MCP):** A modern standard by Anthropic that acts as a universal adapter, connecting agents to data and tools through a structured host-client-server architecture.[50]
- **Agent-to-Agent (A2A) Protocol:** Designed to break down complex workflows into modular units that communicate through standardized handshakes.[50]

Framework	Orchestration Pattern	Key Strength	Typical Use Case
LangGraph	Graph-based (Nodes/Edges)	Predictable flow; state mutability [46]	Complex, cyclic multi-step workflows [47]
AutoGen	Conversation-based	Flexible topologies; chat-driven coordination [54]	Distributed tasks; IT automation [44]
CrewAI	Role-based / Hierarchical	Intuitive human-team mental model [47]	Content generation; cross-functional analysis [47]
OpenAI Swarm	Lightweight / Handoff-based	Simplicity; minimal abstraction [54]	Prototyping; lightweight sales/support [56]

Declarative Agentic Programming: The DSPy Framework

DSPy (Declarative Self-improving Python) represents a shift from "prompt engineering" to "programming" agentic workflows.[58] It separates the logical structure of a program from the specific implementation of its prompts.[58]

Signatures, Modules, and Assertions

In DSPy, the behavior of a module is specified via a Signature—a declarative contract defining input and output behavior.[59] Modules like dspy.ChainOfThought or dspy.ReAct use these signatures to generate intermediate reasoning or tool calls.[59] To ensure reliability, DSPy Assertions (dspy.Assert) and Suggestions (dspy.Suggest) automate the enforcement of constraints.[62] When a constraint is violated, a backtracking mechanism offers the model a chance to self-refine by modifying the signature to include past failures and feedback instructions.[62]

Optimization: MIPROv2 and GEPA

DSPy Optimizers tune program parameters (instructions and few-shot examples) to maximize a user-defined metric.[59]

- **MIPROv2:** Uses Bayesian Optimization to intelligently explore the space of natural-language instructions and demonstrations.[63]
- **GEPA (Genetic-Pareto):** A natural-language reflection optimizer that evolves prompts by analyzing execution traces and reasoning paths.[64] GEPA can optimize prompts per role or sub-agent, making it uniquely suited for complex multi-hop agentic systems.[64]

Systemic Economics, Performance, and Trade-offs

The deployment of agentic AI is as much an economic challenge as it is a technical one.[65] The "Unreliability Tax" refers to the additional costs in compute, latency, and engineering required to mitigate failure risks in probabilistic software stacks.[65]

The Latency-Accuracy Frontier

Agentic workflows significantly outperform single-shot prompting but introduce substantial latency.[65] While a single LLM call might take 800 milliseconds, an orchestrated workflow with a reflection loop can take 10 to 30 seconds.[65] This trade-off is often managed by a Routing Pattern, which classifies query complexity and routes simple requests to fast, single-LLM pipelines and complex ones to deep reasoning agents.[17]

Token Costs and Optimization

The primary economic trap in agent design is quadratic token growth.[65] Because LLMs charge for every input token in every turn, a reflection loop that runs for 10 cycles can consume 50 times the tokens of a linear pass.[65]

Cost Optimization	Technical Mechanism	Benefit

Strategy		
Prompt Caching	Storage and retrieval of static system instructions [65]	90% input cost reduction; 75% latency reduction [65]
ReWOO (Reasoning Without Observation)	Separating plan generation from tool execution to reduce turns [20]	Predictable cost; 70% cost reduction vs pure ReAct [20]
Context Sanitizer	Pruning irrelevant history to prevent context overflow [65]	Maintains precision; prevents token waste [30]
Orchestrated Parallelism	Running independent subtasks concurrently [67]	Reduced overall latency; improved focus per agent [25]

Risks, Governance, and Future Directions

The move toward agentic AI introduces risks that do not exist for static LLMs.[1] Once a model can execute actions—such as modifying code, executing transactions, or navigating the web—hallucinations become concrete failures.[1] Operational risks include erroneous decisions made without human oversight, vulnerability to prompt injection, and the lack of world model consistency.[2]

Governance structures for "Agentic Communities" incorporate formal organizational and ethical rules into inter-agent communication protocols.[9] These communities use accountability mechanisms to ensure verifiable governance when autonomous agents and human participants coordinate.[9] Future research must focus on neuro-symbolic integration to address planning limitations and the development of robust governance models that bridge the "trust gap" between autonomous systems and their human stakeholders.[1]

Synthesis of Agentic Design Methodology

Developing robust agentic systems requires a systematic approach that maps task characteristics to specific patterns.[9] Predictable, repeatable workflows are best served by sequential patterns or ReWOO, while open-ended investigative tasks require the adaptability of ReAct or Reflexion.[17] For highly complex, multi-domain problems, multi-agent collaboration and hierarchical task decomposition provide the necessary modularity and specialized expertise.[17] As the field moves toward standardized frameworks and protocols like MCP and DSPy, the focus shifts from "vibe coding" to principled engineering standards that balance the competing demands of cost, latency, reliability, and autonomy.[17]

Works cited

1. Agentic Artificial Intelligence (AI): Architectures, Taxonomies, and Evaluation of Large Language Model Agents - arXiv, accessed March 6, 2026, https://arxiv.org/html/2601.12560v1
2. (PDF) Agentic artificial intelligence in 2024–2025: technological innovations and application potential in economic applications - ResearchGate, accessed March 6, 2026, https://www.researchgate.net/publication/400310652_Agentic_artificial_intelligence_in_2024-2025_technological_innovations_and_application_potential_in_economic_applications
3. Agentic Design Patterns: A System-Theoretic Framework - arXiv, accessed March 6, 2026, https://arxiv.org/html/2601.19752v1
4. Agentic Design Patterns: A System-Theoretic Framework - OpenReview, accessed March 6, 2026, https://openreview.net/pdf/066e3d13cbc5a82a8c97b1861bb97c4b3cbbb053.pdf
5. LLM and AI Agents for Autonomous Systems: A Survey of Applications, Datasets, and Security Challenges - IEEE Xplore, accessed March 6, 2026, http://ieeexplore.ieee.org/iel8/8784355/11300375/11397656.pdf
6. Agentic AI Across Domains: A Comprehensive Review of Capabilities, Applications, and Future Directions - ResearchGate, accessed March 6, 2026, https://www.researchgate.net/publication/399557210_Agentic_AI_Across_Domains_A_Comprehensive_Review_of_Capabilities_Applications_and_Future_Directions
7. Large Language Model Agents: A Comprehensive Survey on ..., accessed March 6, 2026, https://www.preprints.org/manuscript/202512.2119
8. Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions, accessed March 6, 2026, https://arxiv.org/html/2510.25445v1
9. Architecting Agentic Communities using Design Patterns - arXiv, accessed March 6, 2026, https://arxiv.org/html/2601.03624v1
10. The Belief-Desire-Intention Ontology for modelling mental reality and agency - arXiv, accessed March 6, 2026, https://arxiv.org/html/2511.17162v1
11. BDI Architectures in Intelligent Agents - Emergent Mind, accessed March 6, 2026, https://www.emergentmind.com/topics/bdi-architectures
12. Every AI Agent Architecture in One Place | by Vinayak Talikot | Mar, 2026 | Medium, accessed March 6, 2026, https://medium.com/@vinayak.talikot/every-ai-agent-architecture-in-one-place-595ba68d49cd
13. Agentic AI: A Comprehensive Survey of Technologies, Applications, and Societal Implications | Semantic Scholar, accessed March 6, 2026, https://www.semanticscholar.org/paper/Agentic-AI%3A-A-Comprehensive-Survey-of-Technologies%2C-Pati/bb10ee1cf6d645ad74b9b35bb916086e6b3e340c
14. Understanding AI Agents, Agentic Design Patterns & Use Cases - Sagar Mandal, accessed March 6, 2026, https://www.sagarmandal.com/2024/09/22/understanding-ai-agents-agentic-design-patterns-use-cases/

15. Language Agent Tree Search Unifies Reasoning, Acting, and Planning in Language Models - arXiv.org, accessed March 6, 2026, https://arxiv.org/pdf/2310.04406
16. Design Patterns for Agentic AI and Multi-Agent Systems - AppsTek Corp, accessed March 6, 2026, https://appstekcorp.com/blog/design-patterns-for-agentic-ai-and-multi-agent-systems/
17. A brief look into Agentic Design Patterns - korays.com, accessed March 6, 2026, https://korays.com/a-brief-look-into-agentic-design-patterns/
18. 7 Must-Know Agentic AI Design Patterns - MachineLearningMastery.com, accessed March 6, 2026, https://machinelearningmastery.com/7-must-know-agentic-ai-design-patterns/
19. #12: How Do Agents Learn from Their Own Mistakes? The Role of Reflection in AI, accessed March 6, 2026, https://huggingface.co/blog/Kseniase/reflection
20. Agentic Design Patterns: What They Actually Are (Beyond the Textbooks) | by Rohit Sharma, accessed March 6, 2026, https://levelup.gitconnected.com/agentic-design-patterns-what-they-actually-are-beyond-the-textbooks-fa3eebd01ed8
21. Reflexion | Prompt Engineering Guide, accessed March 6, 2026, https://www.promptingguide.ai/techniques/reflexion
22. Reflexion: Language Agents with Verbal Reinforcement Learning - arXiv, accessed March 6, 2026, https://arxiv.org/html/2303.11366
23. Reflexion - GitHub Pages, accessed March 6, 2026, https://langchain-ai.github.io/langgraph/tutorials/reflexion/reflexion/
24. Agentic AI from First Principles: Reflection | Towards Data Science, accessed March 6, 2026, https://towardsdatascience.com/agentic-ai-from-first-principles-reflection/
25. 7 Practical Design Patterns for Agentic Systems - MongoDB, accessed March 6, 2026, https://www.mongodb.com/resources/basics/artificial-intelligence/agentic-systems
26. Language Agent Tree Search - Emergent Mind, accessed March 6, 2026, https://www.emergentmind.com/topics/language-agent-tree-search-lats
27. awesome-agentic-patterns/patterns/language-agent-tree-search-lats.md at main - GitHub, accessed March 6, 2026, https://github.com/nibzard/awesome-agentic-patterns/blob/main/patterns/language-agent-tree-search-lats.md
28. Language Agent Tree Search Unifies Reasoning Acting and Planning in... - OpenReview, accessed March 6, 2026, https://openreview.net/forum?id=6LNTSrJjBe
29. LanguageAgentTreeSearch | [ICML 2024] Official repository for “Language Agent Tree Search Unifies Reasoning Acting and Planning in Language Models” - GitHub Pages, accessed March 6, 2026, http://lapisrocks.github.io/LanguageAgentTreeSearch/
30. MemGPT: Engineering Semantic Memory through Adaptive Retention and Context Summarization - Information Matters, accessed March 6, 2026,

https://informationmatters.org/2025/10/memgpt-engineering-semantic-memory-through-adaptive-retention-and-context-summarization/
31. [2310.08560] MemGPT: Towards LLMs as Operating Systems - arXiv.org, accessed March 6, 2026, https://arxiv.org/abs/2310.08560
32. MemGPT, accessed March 6, 2026, https://research.memgpt.ai/
33. MemGPT: Towards LLMs as Operating Systems - Leonie Monigatti, accessed March 6, 2026, https://www.leoniemonigatti.com/papers/memgpt.html
34. MemGPT with Real-life Example: Bridging the Gap Between AI and OS | DigitalOcean, accessed March 6, 2026, https://www.digitalocean.com/community/tutorials/memgpt-llm-infinite-context-understanding
35. Toolformer: Language Models Can Teach Themselves to Use Tools - OpenReview, accessed March 6, 2026, https://openreview.net/forum?id=Yacmpz84TH
36. Toolformer: Language Models Can Teach Themselves to Use Tools - Semantic Scholar, accessed March 6, 2026, https://www.semanticscholar.org/paper/Toolformer%3A-Language-Models-Can-Teach-Themselves-to-Schick-Dwivedi-Yu/53d128ea815bcc0526856eb5a9c42cc977cb36a7
37. Toolformer: Language Models Can Teach Themselves to Use Tools - OpenReview, accessed March 6, 2026, https://openreview.net/pdf?id=Yacmpz84TH
38. Toolformer: Language Models Can Teach Themselves to Use Tools - arXiv.org, accessed March 6, 2026, https://arxiv.org/abs/2302.04761
39. Executable Code Actions Elicit Better LLM Agents - arXiv, accessed March 6, 2026, https://arxiv.org/html/2402.01030v1
40. CodeAct Agent Framework - Emergent Mind, accessed March 6, 2026, https://www.emergentmind.com/topics/codeact-agent-framework
41. Executable Code Actions Elicit Better LLM Agents | alphaXiv, accessed March 6, 2026, https://www.alphaxiv.org/overview/2402.01030v4
42. Multi-Agent Systems: Design Patterns and Orchestration - Tetrate, accessed March 6, 2026, https://tetrate.io/learn/ai/multi-agent-systems
43. Multi-Agent Collaboration Mechanisms: A Survey of LLMs - arXiv, accessed March 6, 2026, https://arxiv.org/html/2501.06322v1
44. Agentic AI Frameworks: A Quick Comparison Guide - Arkon Data, accessed March 6, 2026, https://www.arkondata.com/en/post/agentic-ai-frameworks-a-quick-comparison-guide
45. AI agent orchestration for production systems - Redis, accessed March 6, 2026, https://redis.io/blog/ai-agent-orchestration/
46. Top AI Agent Frameworks in 2025 - Codecademy, accessed March 6, 2026, https://www.codecademy.com/article/top-ai-agent-frameworks-in-2025
47. AI Agent Frameworks Compared (2026): LangChain, CrewAI, AutoGen, MetaGPT, OpenDevin + 6 More | Arsum, accessed March 6, 2026, https://arsum.com/blog/posts/ai-agent-frameworks/
48. Towards a science of scaling agent systems: When and why agent systems work, accessed March 6, 2026,

https://research.google/blog/towards-a-science-of-scaling-agent-systems-when-and-why-agent-systems-work/
49. How to design multi-agent communication protocols in intelligent agent development?, accessed March 6, 2026, https://www.tencentcloud.com/techpedia/126131
50. What Are AI Agent Protocols? - IBM, accessed March 6, 2026, https://www.ibm.com/think/topics/ai-agent-protocols
51. Agent Communication Protocols Explained | DigitalOcean, accessed March 6, 2026, https://www.digitalocean.com/community/tutorials/agent-communication-protocols-explained
52. AI Agent Protocols Explained: The New Language of Intelligent Systems, accessed March 6, 2026, https://medium.com/@ajinkyachatufale/ai-agent-protocols-explained-the-new-language-of-intelligent-systems-681c5855880a
53. Multi-Agent Systems: Orchestrating AI Agents with A2A Protocol | by Yusuf Baykaloğlu | Jan, 2026, accessed March 6, 2026, https://medium.com/@yusufbaykaloglu/multi-agent-systems-orchestrating-ai-agents-with-a2a-protocol-19a27077aed8
54. AutoGen vs. CrewAI vs. LangGraph vs. OpenAI Multi-Agents Framework - Galileo AI, accessed March 6, 2026, https://galileo.ai/blog/autogen-vs-crewai-vs-langgraph-vs-openai-agents-framework
55. Best Agentic AI Frameworks Compared: Top Picks & Feature Breakdown - Riseup Labs, accessed March 6, 2026, https://riseuplabs.com/best-agentic-ai-frameworks-compared/
56. A Detailed Comparison of Top 6 AI Agent Frameworks in 2026 - Turing, accessed March 6, 2026, https://www.turing.com/resources/ai-agent-frameworks
57. OpenAI Agents SDK vs LangGraph vs Autogen vs CrewAI - Composio, accessed March 6, 2026, https://composio.dev/blog/openai-agents-sdk-vs-langgraph-vs-autogen-vs-crewai
58. DSPy, accessed March 6, 2026, https://dspy.ai/
59. DSPy: The Future of AI Programming — A Complete Guide | by Mahasri Seenivasan, accessed March 6, 2026, https://medium.com/@mahasris0304/dspy-the-future-of-ai-programming-a-complete-guide-9caeb9882c82
60. Signatures - DSPy, accessed March 6, 2026, https://dspy.ai/learn/programming/signatures/
61. DSPy: Build and Optimize Agentic Apps - DeepLearning.AI, accessed March 6, 2026, https://www.deeplearning.ai/short-courses/dspy-build-optimize-agentic-apps/
62. DSPy Assertions, accessed March 6, 2026, https://dspy.ai/learn/programming/7-assertions/
63. Optimizers - DSPy, accessed March 6, 2026,

https://dspy.ai/learn/optimization/optimizers/
64. GEPA: The Game-Changing DSPy Optimizer for Agentic AI - Superagentic AI, accessed March 6, 2026, https://super-agentic.ai/resources/super-posts/gepa-game-changing-dspy-optimizer-agentic-ai/
65. The Hidden Economics of AI Agents: Managing Token Costs and Latency Trade-offs, accessed March 6, 2026, https://online.stevens.edu/blog/hidden-economics-ai-agents-token-costs-latency/
66. Agentic AI Design Patterns: ReAct, ReWOO, CodeAct, and Beyond - Capabl, accessed March 6, 2026, https://capabl.in/blog/agentic-ai-design-patterns-react-rewoo-codeact-and-beyond
67. Agentic Design Patterns | Terezinha Tech Operations (ttoss), accessed March 6, 2026, https://ttoss.dev/docs/ai/agentic-design-patterns
68. A Survey on Large Language Model based Human-Agent Systems - arXiv, accessed March 6, 2026, https://arxiv.org/html/2505.00753v1
69. [2510.25445] Agentic AI: A Comprehensive Survey of Architectures, Applications, and Future Directions - arXiv.org, accessed March 6, 2026, https://arxiv.org/abs/2510.25445
70. Choose a design pattern for your agentic AI system | Cloud Architecture Center, accessed March 6, 2026, https://docs.cloud.google.com/architecture/choose-design-pattern-agentic-ai-system
71. Agentic AI: One Year After Andrew Ng's Design Patterns – Hype or Reality? Part 1 - Medium, accessed March 6, 2026, https://medium.com/@haileyq/agentic-ai-one-year-after-andrew-ngs-design-patterns-hype-or-reality-6fbd87dbe870